Production MAINHEAD Backfill — Operator Runbook

Step-by-step operator procedure to populate `SiteVisit.mainheadId` from the legacy `SiteVisit.mainhead` text column, restoring QA visibility under Governance G3. **Runbook only — do not execute. Run in an approved maintenance window.**

Change ID: `governance-g3-backfill-sitevisit-mainhead-20260604` | Target DB: `ascure` (container `ascure-postgres`, PostgreSQL 16) | Properties: idempotent, ambiguity-guarded, reversible.

GOLDEN RULE — Do not run Section 4 until BOTH Section 3 (safe values) and Section 7 (Go/No-Go) are satisfied. The Section 2 pre-checks are read-only and must run inside a read-only session.

1. Connect to production PostgreSQL

Run from the production host (where `docker-compose.prod.yml` lives). Port 5432 is not published to the host, so connect inside the container.

```
# 1a. Confirm the database container is the expected one
docker ps --filter "name=ascure-postgres" \
  --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
# Expect: ascure-postgres postgres:16-alpine Up ...

# 1b. Open an interactive psql session (port 5432 is NOT host-published)
docker compose -f docker-compose.prod.yml exec postgres \
  psql -U ascure -d ascure
# Fallback: docker exec -it ascure-postgres psql -U ascure -d ascure
```

Inside psql, pin safety flags and confirm you are on production:

```
\set ON_ERROR_STOP on
\timing on
\conninfo -- confirm: database "ascure" as user "ascure"
SELECT current_database(), inet_server_addr(), version();

-- Identity sanity check: ADMIN-visible total must match production (7).
SELECT count(*) AS admin_total_visits FROM "SiteVisit";
-- STOP if this is not 7 (or your expected total) -- you are on the wrong DB.
```

2. Queries to run BEFORE backfill (read-only)

Make the pre-check session physically unable to write:

```
SET default_transaction_read_only = on; -- hard guard for all of section 2
\set qa_email 'qa.manager@ascure.local' -- <- set to the QA Manager's real email
```

2a. Optional physical safety net — table snapshot (host shell)

```
docker exec ascure-postgres \
  pg_dump -U ascure -d ascure -t '"SiteVisit"' --data-only \
  -f /tmp/SiteVisit_pre_backfill_20260604.sql
docker cp ascure-postgres:/tmp/SiteVisit_pre_backfill_20260604.sql \
  ./SiteVisit_pre_backfill_20260604.sql
```

2b. NULL-FK volume

```
-- B1: NULL-FK volume (record all three numbers)
SELECT
  count(*) AS total_visits,
  count(*) FILTER (WHERE "mainheadId" IS NULL) AS null_fk_visits,
  count(*) FILTER (WHERE "mainheadId" IS NOT NULL) AS populated_fk_visits
FROM "SiteVisit";
```

2c. Planned coverage

```
-- B2: planned coverage (drives the Go/No-Go)
WITH legacy AS (
  SELECT sv.id, lower(btrim(regexp_replace(sv."mainhead",'\s+',' ','g'))) AS norm_text
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
cand AS (
  SELECT l.id, count(DISTINCT m.id) AS mc
  FROM legacy l
  LEFT JOIN "Mainhead" m ON m."isActive" AND (
       lower(btrim(regexp_replace(m.name,'\s+',' ','g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code,''),'\s+',' ','g'))) = l.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code,'')||' - '||m.name,'\s+',' ','g'))) = l.norm_text)
  GROUP BY l.id
)
SELECT
  count(*) FILTER (WHERE mc = 1) AS will_backfill,
  count(*) FILTER (WHERE mc > 1) AS ambiguous_skip,
  count(*) FILTER (WHERE mc = 0) AS unmatched_skip
FROM cand;
```

2d. The mapping — review every line

```
-- D2: distinct legacy-text -> Mainhead with decision (review EVERY row by eye)
WITH legacy AS (
  SELECT sv."mainhead" AS legacy_text,
         lower(btrim(regexp_replace(sv."mainhead",'\s+',' ','g'))) AS norm_text
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
distinct_text AS (
  SELECT legacy_text, norm_text, count(*) AS affected_rows
  FROM legacy GROUP BY legacy_text, norm_text
),
matched AS (
  SELECT d.legacy_text, m.id AS mainhead_id, m.code, m.name
  FROM distinct_text d
  JOIN "Mainhead" m ON m."isActive" AND (
       lower(btrim(regexp_replace(m.name,'\s+',' ','g'))) = d.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code,''),'\s+',' ','g'))) = d.norm_text
    OR lower(btrim(regexp_replace(coalesce(m.code,'')||' - '||m.name,'\s+',' ','g'))) = d.norm_text)
)
SELECT
  d.legacy_text, d.affected_rows,
  count(DISTINCT m.mainhead_id) AS distinct_matches,
  string_agg(DISTINCT m.code || ' / ' || m.name, '; ') AS matched_mainheads,
  CASE WHEN count(DISTINCT m.mainhead_id)=1 THEN 'WILL_BACKFILL'
       WHEN count(DISTINCT m.mainhead_id)>1 THEN 'AMBIGUOUS_SKIP'
       ELSE 'UNMATCHED_SKIP' END AS decision
FROM distinct_text d
LEFT JOIN matched m ON m.legacy_text = d.legacy_text
GROUP BY d.legacy_text, d.affected_rows
ORDER BY decision, d.affected_rows DESC;
```

2e. QA baseline (self-resolving — no UUID copy-paste)

```
-- B3: what the QA Manager sees today (mirrors buildScopeContext)
WITH qa_user AS (SELECT id FROM "User" WHERE email = :'qa_email'),
qa_mainheads AS (
  SELECT uma."mainheadId" AS id
  FROM "UserMainheadAccess" uma JOIN "Mainhead" m ON m.id=uma."mainheadId" AND m."isActive"
  WHERE uma."userId" IN (SELECT id FROM qa_user)
  UNION
  SELECT m.id
  FROM "UserOperationalRegionAccess" ura
  JOIN "OperationalRegion" r ON r.id=ura."operationalRegionId" AND r."isActive"
  JOIN "Mainhead" m ON m."operationalRegionId"=ura."operationalRegionId" AND m."isActive"
  WHERE ura."userId" IN (SELECT id FROM qa_user)
)
SELECT
  (SELECT count(*) FROM qa_mainheads) AS qa_mainhead_count,
  (SELECT count(*) FROM "SiteVisit"
     WHERE "mainheadId" IN (SELECT id FROM qa_mainheads)) AS qa_visible_before;

RESET default_transaction_read_only; -- end the read-only session before section 4
```

3. What output values are SAFE to proceed

Record each value from Section 2 and confirm its safe condition:

Value (source)	Safe-to-proceed condition
`admin_total_visits` (1)	Equals the known production total (**7**). If not, wrong DB -> **abort**.
`null_fk_visits` (B1)	`> 0`. If 0, nothing to backfill -> stop.
`will_backfill` (B2)	`>= 1` and equals the number of rows you intend to fix.
`ambiguous_skip` (B2/D2)	Reviewed. Each ambiguous text is accepted-as-skipped or resolved manually first. Never run blind over rows you actually need.
`unmatched_skip` (B2/D2)	Reviewed and accepted (these stay NULL; need data cleanup, not this backfill).
D2 every `WILL_BACKFILL` row	Maps to the **intended** Mainhead on inspection (esp. KL BARAT -> KLB / KL BARAT).
`qa_mainhead_count` (B3)	`>= 1` (QA Manager actually has MAINHEAD access).
`qa_visible_before` (B3)	Recorded (expected **1**) — the post-run comparison anchor.
Snapshot + window	Physical snapshot (2a) captured; maintenance window + change approval active.

Proceed only if every row above is in its safe state. Otherwise -> No-Go (Section 7).

4. The exact backfill transaction

Open a **fresh write session**, set `\set ON_ERROR_STOP on`, and paste the whole block as one unit. It self-aborts on any integrity failure (3d) — if you see `COMMIT`, it succeeded.

```
\set ON_ERROR_STOP on
BEGIN;

-- 3a. Backup / audit table (forward record + rollback source). Survives the transaction.
CREATE TABLE IF NOT EXISTS "_backfill_SiteVisit_mainhead_20260604" (
  site_visit_id uuid PRIMARY KEY,
  previous_mainhead_id uuid, -- always NULL for these rows; captured for audit
  legacy_mainhead_text text,
  assigned_mainhead_id uuid NOT NULL,
  backfilled_at timestamptz NOT NULL DEFAULT now()
);

-- 3b. Resolve the unique-match set (only 1-distinct-candidate rows qualify).
WITH legacy AS (
  SELECT sv.id, sv."mainhead" AS legacy_text,
         lower(btrim(regexp_replace(sv."mainhead", '\s+', ' ', 'g'))) AS norm_text
  FROM "SiteVisit" sv
  WHERE sv."mainheadId" IS NULL
    AND sv."mainhead" IS NOT NULL AND btrim(sv."mainhead") <> ''
),
candidate AS (
  SELECT l.id, l.legacy_text, m.id AS mainhead_id
  FROM legacy l
  JOIN "Mainhead" m
    ON m."isActive" = TRUE
   AND (
        lower(btrim(regexp_replace(m.name,'\s+',' ','g'))) = l.norm_text
     OR lower(btrim(regexp_replace(coalesce(m.code,''),'\s+',' ','g'))) = l.norm_text
     OR lower(btrim(regexp_replace(coalesce(m.code,'')||' - '||m.name,'\s+',' ','g'))) = l.norm_text
   )
),
resolved AS (
  SELECT id, legacy_text,
         count(DISTINCT mainhead_id) AS match_count,
         min(mainhead_id) AS mainhead_id -- meaningful only when match_count = 1
  FROM candidate GROUP BY id, legacy_text
)
INSERT INTO "_backfill_SiteVisit_mainhead_20260604"
```

```
       (site_visit_id, previous_mainhead_id, legacy_mainhead_text, assigned_mainhead_id)
SELECT r.id, NULL::uuid, r.legacy_text, r.mainhead_id
FROM resolved r
WHERE r.match_count = 1
ON CONFLICT (site_visit_id) DO NOTHING; -- re-run safe

-- 3c. Apply the backfill ONLY to captured rows, ONLY while still NULL.
-- updatedAt is intentionally NOT modified (preserves last-activity / staleness signals).
UPDATE "SiteVisit" sv
SET "mainheadId" = b.assigned_mainhead_id
FROM "_backfill_SiteVisit_mainhead_20260604" b
WHERE sv.id = b.site_visit_id
  AND sv."mainheadId" IS NULL; -- idempotency guard

-- 3d. In-transaction sanity check: every touched row points to an ACTIVE Mainhead.
DO $$
DECLARE bad int;
BEGIN
  SELECT count(*) INTO bad
  FROM "_backfill_SiteVisit_mainhead_20260604" b
  JOIN "SiteVisit" sv ON sv.id = b.site_visit_id
  LEFT JOIN "Mainhead" m ON m.id = sv."mainheadId" AND m."isActive" = TRUE
  WHERE sv."mainheadId" <> b.assigned_mainhead_id -- diverged from plan
     OR m.id IS NULL; -- missing/inactive Mainhead
  IF bad > 0 THEN
    RAISE EXCEPTION 'Backfill integrity check failed for % row(s); rolling back', bad;
  END IF;
END $$;

COMMIT;
```

If anything other than `COMMIT` appears (e.g. the `RAISE EXCEPTION`), the transaction has already rolled itself back. **Stop, do not retry blindly** — return to Section 2, re-diagnose, then re-run Sections 3/7.

5. Queries to run AFTER backfill (read-only)

Re-use the same `\set qa_email` value from Section 2.

```
-- A1: rows actually changed (must equal B2.will_backfill)
SELECT count(*) AS rows_backfilled FROM "_backfill_SiteVisit_mainhead_20260604";

-- A2: NULL volume after (must equal B1.null_fk_visits - A1.rows_backfilled)
SELECT count(*) FILTER (WHERE "mainheadId" IS NULL) AS null_fk_after FROM "SiteVisit";

-- A3: referential + activity integrity (must be 0)
SELECT count(*) AS broken_or_inactive
FROM "SiteVisit" sv
LEFT JOIN "Mainhead" m ON m.id = sv."mainheadId"
WHERE sv."mainheadId" IS NOT NULL AND (m.id IS NULL OR m."isActive" = FALSE);

-- A4: every backfilled row matches its plan exactly (must be 0)
SELECT count(*) AS diverged
FROM "_backfill_SiteVisit_mainhead_20260604" b
JOIN "SiteVisit" sv ON sv.id = b.site_visit_id
WHERE sv."mainheadId" <> b.assigned_mainhead_id;

-- A5: QA visibility now (re-resolves QA mainheads; compare to B3.qa_visible_before)
WITH qa_user AS (SELECT id FROM "User" WHERE email = :'qa_email'),
qa_mainheads AS (
  SELECT uma."mainheadId" AS id
  FROM "UserMainheadAccess" uma JOIN "Mainhead" m ON m.id=uma."mainheadId" AND m."isActive"
  WHERE uma."userId" IN (SELECT id FROM qa_user)
  UNION
  SELECT m.id
  FROM "UserOperationalRegionAccess" ura
  JOIN "OperationalRegion" r ON r.id=ura."operationalRegionId" AND r."isActive"
  JOIN "Mainhead" m ON m."operationalRegionId"=ura."operationalRegionId" AND m."isActive"
  WHERE ura."userId" IN (SELECT id FROM qa_user)
)
SELECT count(*) AS qa_visible_after
FROM "SiteVisit" WHERE "mainheadId" IN (SELECT id FROM qa_mainheads);
```

Then confirm in the live app: log in as the QA Manager and verify the Site Visits list and Dashboard counts increased as expected.

6. Rollback command

Exact reversal — touches only rows this backfill set, and only if they still hold the assigned value (never clobbers a later legitimate edit).

```
\set ON_ERROR_STOP on
BEGIN;
UPDATE "SiteVisit" sv
SET "mainheadId" = b.previous_mainhead_id -- NULL
FROM "_backfill_SiteVisit_mainhead_20260604" b
WHERE sv.id = b.site_visit_id
  AND sv."mainheadId" = b.assigned_mainhead_id; -- guard: only undo our own writes
COMMIT;

-- Verify: should return rows_backfilled again, all now NULL
SELECT count(*) AS reverted_to_null
FROM "_backfill_SiteVisit_mainhead_20260604" b
JOIN "SiteVisit" sv ON sv.id = b.site_visit_id
WHERE sv."mainheadId" IS NULL;

-- Cleanup (post-stabilization only -- keep through the observation window):
-- DROP TABLE "_backfill_SiteVisit_mainhead_20260604";
```

Physical fallback (last resort, if the audit table is unusable): restore `./SiteVisit_pre_backfill_20260604.sql` from step 2a per your DR procedure.

7. Go / No-Go criteria

GO — proceed to Section 4 only if ALL are true

- Section 1 identity confirmed: `current_database() = ascure` and `admin_total_visits` = expected (7).
- `null_fk_visits > 0`.
- `will_backfill >= 1` and equals the count you intend to fix.
- Every D2 `WILL_BACKFILL` row maps to the intended Mainhead.
- `ambiguous_skip` and `unmatched_skip` each reviewed and explicitly accepted (or pre-resolved).
- `qa_mainhead_count >= 1`; `qa_visible_before` recorded.
- Physical snapshot (2a) captured; window + approval active.

NO-GO — do not run Section 4 if ANY is true

- Wrong/uncertain database, or `admin_total_visits` not equal to expected.
- `will_backfill = 0` (investigate text shape; do not force).
- Any `WILL_BACKFILL` mapping looks wrong, or a needed row sits in `AMBIGUOUS_SKIP` unresolved.
- Pre-existing integrity problems surface (resolve first).
- No snapshot, no rollback path, or no approval/window.

POST-RUN verdict — decide immediately after Section 5

SUCCESS — keep the change if all hold:

- `rows_backfilled == will_backfill`
- `null_fk_after == null_fk_visits - rows_backfilled`
- `broken_or_inactive == 0`
- `diverged == 0`
- `qa_visible_after` = `qa_visible_before` + (backfilled rows resolving to QA-accessible mainheads), and the QA Manager's app view reflects it.

FAIL — execute Section 6 rollback now if any after-check fails, the `COMMIT` did not print, or live QA visibility is wrong. Then re-diagnose before any retry.